

SHOPMART E-COMMERCE PROJECT

– Practical Logbook

WEEK 6: Database Integration & CRUD Operations

Student	Stephen Otiende
Reg. No.	BBIT/2018/36596
Course	BIT3208 – Advanced Web Design and Development
Supervisor	Mr. Michael Nyoro
Week	Week 6
Focus Area	Database Integration & CRUD Operations in ShopMart

1. Learning Objectives

By the end of this week, I was expected to be able to:

- Understand what database integration means and why it is important in web development
- Know the four CRUD operations — Create, Read, Update, and Delete — and how they work
- Connect my ShopMart PHP project to a MySQL database using the MySQLi extension
- Write PHP code to add new products, display them, edit their details, and remove them
- Build forms that users can fill in to enter product data into the database
- Test my work to make sure data was being saved, retrieved, and changed correctly

2. What I Learned This Week

2.1 What is Database Integration?

Before this week, I understood that websites can display information. What I learned is that most real websites do not just display fixed content — they store and retrieve information from a database every time a page loads. This is called database integration.

For example, when a customer visits ShopMart and sees a list of products, those products are not typed into the HTML manually. They are pulled from a MySQL database by PHP. This is what makes the site dynamic — the content changes based on what is stored in the database.

Some real-world examples I relate to:

- Online shopping sites like Jumia — products come from a database
- University portals like MKU Online — student results are stored in a database
- M-PESA transaction history — every transaction is a record in a database
- Social media like Facebook — posts, likes, and comments are all database records

2.2 CRUD Operations — The Four Things You Can Do With Data

CRUD stands for the four basic operations you can perform on any database record. I think of it like managing a physical file cabinet: you can add a new file (Create), read an existing one (Read), update it (Update), or throw it away (Delete).

Operation	SQL Command	ShopMart Example	PHP Tool Used
Create	<i>INSERT INTO</i>	Admin adds a new product to the shop	<i>prepare() + bind_param()</i>
Read	<i>SELECT</i>	Customer browses the product listing page	<i>mysqli_fetch_assoc()</i>
Update	<i>UPDATE ... SET</i>	Admin changes a product price or stock level	<i>prepare() + bind_param()</i>
Delete	<i>DELETE FROM</i>	Admin removes a product that is no longer sold	<i>prepare() + bind_param()</i>

3. What I Built — ShopMart CRUD Features

3.1 Setting Up the Database

The first thing I did was open phpMyAdmin on XAMPP and create a database called shopmart. Inside it, I created a products table to store all the information about each item being sold in the shop. I added columns for the product name, description, price, how many are in stock, and which category it belongs to.

```
-- I ran this in phpMyAdmin to create my database and products table
CREATE DATABASE IF NOT EXISTS shopmart;
USE shopmart;

CREATE TABLE IF NOT EXISTS products (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(150) NOT NULL,
  description TEXT,
  price       DECIMAL(10,2) NOT NULL,
  stock_qty   INT          DEFAULT 0,
  category    VARCHAR(100),
  created_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);
```

The id column uses AUTO_INCREMENT so MySQL automatically gives each product a unique number. I do not have to enter it manually every time I add a product.

3.2 Connecting PHP to MySQL — config.php

I created a file called config.php to hold the database connection. I include this file at the top of every other PHP file that needs to talk to the database, so I do not have to repeat the connection code everywhere.

```
<?php
// This file connects my PHP project to the MySQL database
// I include it at the top of any file that needs to use the database

$conn = mysqli_connect("localhost", "root", "", "shopmart");

if (!$conn) {
    die("Connection Failed: " . mysqli_connect_error());
}
?>
```

The four things I pass to `mysqli_connect()` are: the server name (localhost because XAMPP runs on my own computer), the username (root is the default in XAMPP), the password (empty by default), and the database name (shopmart).

3.3 CREATE — Adding a New Product

I built a form in `admin/add_product.php` where the admin can type in product details and click a button to save them. When the form is submitted, my PHP code takes the values, checks that they are valid, and inserts a new row into the products table.

One important thing I learned is that I should never put form values directly into a SQL query. This is called SQL injection and it is dangerous. Instead, I used prepared statements, which keep the query and the data separate so the database cannot be tricked.

```
<?php
include('../config.php');

if (isset($_POST['add_product'])) {

    // Step 1: Get the values from the form and clean them
    $name = htmlspecialchars(trim($_POST['name']));
    $desc = htmlspecialchars(trim($_POST['description']));
    $price = floatval($_POST['price']); // Make sure price is a decimal number
    $stock = intval($_POST['stock_qty']); // Make sure stock is a whole number
    $cat = htmlspecialchars(trim($_POST['category']));

    // Step 2: Prepare the SQL query using ? as placeholders
    $stmt = $conn->prepare(
        "INSERT INTO products (name, description, price, stock_qty, category)
        VALUES (?, ?, ?, ?, ?)"
    );

    // Step 3: Bind the actual values to the placeholders
```

```
// "ssdis" means: string, string, decimal, integer, string
$stmt->bind_param("ssdis", $name, $desc, $price, $stock, $cat);

// Step 4: Run the query and redirect if it worked
if ($stmt->execute()) {
    header("Location: manage_products.php?added=1");
    exit();
}
}
?>
```

3.4 READ — Displaying Products on the Shop Page

In products.php, I wrote code to fetch all products from the database and display them as cards on the page. I used a while loop to go through each row returned by the query and print the product's name, price, category, and stock count.

```
<?php
include('config.php');

// Fetch all products that are still in stock, newest first
$result = mysqli_query($conn,
    "SELECT * FROM products WHERE stock_qty > 0 ORDER BY created_at DESC"
);

// Loop through each product and display it
while ($row = mysqli_fetch_assoc($result)) {
    echo '<div class="product-card">';
    // htmlspecialchars() makes sure product names cannot contain harmful HTML
    echo '<h3>' . htmlspecialchars($row['name']) . '</h3>';
    echo '<p>KES ' . number_format($row['price'], 2) . '</p>';
    echo '<p>In Stock: ' . (int)$row['stock_qty'] . '</p>';
    echo '</div>';
}
?>
```

I used WHERE stock_qty > 0 so that customers only see products that are actually available. Products with zero stock are hidden from the public listing.

3.5 UPDATE — Editing an Existing Product

The edit page (admin/edit_product.php) works in two steps. First, when the admin clicks Edit on a product, the page loads with the form already filled in with that product's current details. Second, when the admin changes something and submits the form, the PHP code updates that row in the database.

```
<?php
include('../config.php');

// Get the product ID from the URL, e.g. edit_product.php?id=3
$id = intval($_GET['id']);

// Step 1: Fetch the current details of this product to pre-fill the form
```

```
$stmt = $conn->prepare("SELECT * FROM products WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$product = $stmt->get_result()->fetch_assoc();

// Step 2: When the admin submits the updated form, save the changes
if (isset($_POST['update_product'])) {
    $name = htmlspecialchars(trim($_POST['name']));
    $price = floatval($_POST['price']);
    $stock = intval($_POST['stock_qty']);

    $upd = $conn->prepare(
        "UPDATE products SET name = ?, price = ?, stock_qty = ? WHERE id = ?"
    );
    // The WHERE id = ? part is very important – it makes sure only THIS product
    // is updated
    $upd->bind_param("sdii", $name, $price, $stock, $id);
    $upd->execute();

    header("Location: manage_products.php?updated=1");
    exit();
}
?>
```

The WHERE id = ? part in the UPDATE query is critical. Without it, every single product in the database would be updated with the same values, which would destroy all the other records.

3.6 DELETE — Removing a Product

I built the delete feature in admin/delete_product.php. One thing I was taught is that delete operations should only happen through a POST form, not through a link (GET request). This is because a link can be clicked accidentally or even by a search engine crawler, which could delete products without the admin meaning to.

```
<?php
include('../config.php');

// Refuse the request if it is not coming from a form submission (POST)
// This protects against accidental deletion via a browser link
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header("Location: manage_products.php");
    exit();
}

// Get and validate the product ID
$id = intval($_POST['id']);

// Delete only the product with this specific ID
$stmt = $conn->prepare("DELETE FROM products WHERE id = ?");
$stmt->bind_param("i", $id);

if ($stmt->execute()) {
    header("Location: manage_products.php?deleted=1");
}
exit();
?>
```

4. Good Coding Practices I Applied

While building the CRUD features this week, I made sure to follow some important practices that I learned in class:

Practice	Why I Did It	How I Applied It in ShopMart
Prepared Statements	To stop SQL injection — where someone tricks the database with malicious input	<i>Used <code>prepare()</code> and <code>bind_param()</code> on every INSERT, UPDATE, and DELETE query</i>
Input Validation	To make sure the data entered in forms is in the right format before saving	<i>Used <code>intval()</code> for stock, <code>floatval()</code> for price, and <code>empty()</code> to check required fields</i>
Output Encoding	To stop XSS — where someone puts harmful HTML or JavaScript into form fields	<i>Used <code>htmlspecialchars()</code> on every value I echo onto the page</i>
POST for Delete	To prevent products from being deleted by accident through a URL or link click	<i>The delete button is inside a form with <code>method=POST</code>, not a plain link</i>
Feedback Messages	To let the admin know whether their action worked or failed	<i>After each action, the page redirects with <code>?added=1</code>, <code>?updated=1</code>, or <code>?deleted=1</code></i>

5. What I Did Each Day

Day	What I Did	Result
Day 1	I read through the Week 6 notes on CRUD and database integration. I then opened phpMyAdmin on XAMPP and created the shopmart database and the products table with all the right columns.	<i>The database and table were created successfully. I inserted a few test rows manually in phpMyAdmin to confirm the table was working.</i>
Day 2	I created config.php with the MySQLi connection code. I then included it in a test PHP file and loaded the page on localhost to confirm that PHP could connect to MySQL without errors.	<i>The connection was successful. The page loaded without any error messages.</i>
Day 3	I built admin/add_product.php — the HTML form and the PHP handler. I tested it by filling in the form and submitting it, then checking phpMyAdmin to see if the new product appeared in the table.	<i>Products added through the form appeared correctly in the database.</i>
Day 4	I built products.php to display products on the public shop page, and admin/edit_product.php so the admin can update product details. I checked that the	<i>The shop page displayed all products from the database. The edit form pre-filled correctly</i>

	edit form loaded with the correct existing values pre-filled.	<i>and saved changes successfully.</i>
Day 5	I built admin/delete_product.php and connected the Delete button in manage_products.php to it using a POST form. I tested the full CRUD cycle from start to finish and wrote up the logbook entry.	<i>All four CRUD operations worked correctly. Feedback messages appeared after each action.</i>

6. Reflection — What I Learned and Found Challenging

6.1 Answers to the Weekly Reflection Questions

Question	My Answer
Why are databases important in web applications?	Without a database, a website can only show the same fixed information to everyone. A database allows the site to store information permanently and show different content to different users. For ShopMart, the database stores all the products, so the admin can add or remove items without editing the HTML code every time.
What is the difference between static and dynamic websites?	A static website has the same content for everyone and never changes unless a developer edits the HTML file manually. A dynamic website like ShopMart pulls content from a database, so different pages can show different information depending on what is stored. For example, the product listing page always shows the latest products without anyone having to edit the code.
Explain CRUD operations with examples.	CRUD stands for Create, Read, Update, and Delete. In ShopMart: Create means the admin adds a new product using the add form. Read means a customer views the list of available products. Update means the admin edits a product's price when it changes. Delete means the admin removes a product that is no longer being sold.
How does PHP communicate with MySQL?	PHP communicates with MySQL through the MySQLi extension. I use mysqli_connect() to open a connection, then prepare() and bind_param() to build a safe query, execute() to run it, and fetch_assoc() to get the results back as an array I can use in my code.
Why should developers validate user input?	If I do not validate input, someone could enter wrong data such as a negative price or a letter where a number is expected, which would corrupt the database. Even worse, without validation someone could attempt an SQL injection attack. Validating input makes sure only correct, safe data is stored.
What security risks can arise from poor database design?	If I write queries without prepared statements, an attacker can inject their own SQL code into the query and read or delete data they should not be able to access. If I do not check access rights, any user could reach the admin panel.

Poor database design also leads to data being duplicated or lost due to missing constraints like foreign keys.

6.2 Key Things I Learned

- I used to think SQL injection was just a theory, but after building the forms I understood how easy it would be to break an unprotected query. Prepared statements completely remove that risk by separating the query from the data.
- Using `intval()` and `floatval()` is a simple habit but it makes a big difference — if someone types letters into the price field, PHP converts it to zero rather than letting bad data reach the database.
- I learned why DELETE must use POST and not GET. A GET request can be triggered just by someone clicking a link or even by Google's search crawler visiting the page, which could accidentally delete records.
- The `number_format()` function was something new for me. It made prices display as 3,500.00 instead of 3500, which looks much more professional on the shop page.

6.3 Difficulties I Encountered

- The first time I tested my connection, the page showed a blank error. I spent time checking the code before realising the database name in `config.php` did not match the one I created in phpMyAdmin. I fixed it by double-checking the exact name.
- When I first wrote the UPDATE query, I forgot to include `WHERE id = ?` and accidentally updated all the products in the table with the same values. I had to re-insert the sample data and add the WHERE clause.
- I did not know at first how to show a success message after a redirect. I learned to append a query string like `?added=1` to the URL and check for it on the destination page using `$_GET`.

7. References

PHP.net (2024) `mysqli::prepare` — Manual. Available at: <https://www.php.net/manual/en/mysqli.prepare.php> (Accessed: June 2026).

MySQL (2024) SQL Syntax — INSERT, SELECT, UPDATE, DELETE. Available at: <https://dev.mysql.com/doc/refman/8.0/en/sql-statements.html> (Accessed: June 2026).

OWASP Foundation (2021) SQL Injection Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (Accessed: June 2026).

Welling, L. and Thomson, L. (2016) PHP and MySQL Web Development. 5th edn. Addison-Wesley Professional.